CA4

- Johannes Husevåg Standal
- Rozalia Anna Ciupka

Imports

In [3]:
```
# Data handling
import numpy as np
import pandas as pd

# Visualisation
import seaborn as sns
import matplotlib.pyplot as plt

# Preprocessing
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

# Pipelines and dimension reduction
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier

# Model selection
from sklearn.model_selection import train_test_split, GridSearchCV

# Evaluation
from sklearn.metrics import f1_score, accuracy_score, ConfusionMatrixDisplay, confu
```

Read the data

In [4]:
```
df_train = pd.read_csv("trainCA4DAT200.csv")
df_submission = pd.read_csv("testCA4DAT200.csv")
```

Data exploration and visualisation

In [5]:
```
# Check feature data types
df_train.info()

"""
We can see that the dataset contains both numerical and categorical features.
```

```
The categorical features being: State, International plan, and Voice mail plan.
there is also a sneaky categorial feature for AreaCode, which is a numerical featur
"""

categorical_features = [
    "State",
    "AreaCode",
    "InternationalPlan",
    "VoiceMailPlan"
]
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3500 entries, 0 to 3499
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   CustomerId            3500 non-null   int64  
 1   State                 3500 non-null   object 
 2   AccountLength         3500 non-null   int64  
 3   AreaCode              3500 non-null   int64  
 4   InternationalPlan     3500 non-null   object 
 5   VoiceMailPlan         3500 non-null   object 
 6   NumVoiceMailMessages  3500 non-null   int64  
 7   DayMinutes            3500 non-null   float64
 8   DayCalls              3500 non-null   int64  
 9   DayCharge             3500 non-null   float64
 10  EveMinutes            3500 non-null   float64
 11  EveCalls              3500 non-null   int64  
 12  EveCharge             3500 non-null   float64
 13  NightMinutes          3500 non-null   float64
 14  NightCalls            3500 non-null   int64  
 15  NightCharge           3500 non-null   float64
 16  IntlMinutes           3500 non-null   float64
 17  IntlCalls             3500 non-null   int64  
 18  IntlCharge            3500 non-null   float64
 19  CustomerServiceCalls  3500 non-null   int64  
 20  Churn                 3500 non-null   int64  
dtypes: float64(8), int64(10), object(3)
memory usage: 574.3+ KB
```

In [6]:
```
# Check for missing values, we have none
print(f"missing values in training data: {df_train.isna().sum()}")
```

```
missing values in training data: CustomerId              0
State                   0
AccountLength           0
AreaCode                0
InternationalPlan       0
VoiceMailPlan           0
NumVoiceMailMessages    0
DayMinutes              0
DayCalls                0
DayCharge               0
EveMinutes              0
EveCalls                0
EveCharge               0
NightMinutes            0
NightCalls              0
NightCharge             0
IntlMinutes             0
IntlCalls               0
IntlCharge              0
CustomerServiceCalls    0
Churn                   0
dtype: int64
```

In [5]:
```
# Check for duplicate rows, we have none
print("Duplicate rows:", df_train.duplicated().sum())
```

```
Duplicate rows: 0
```

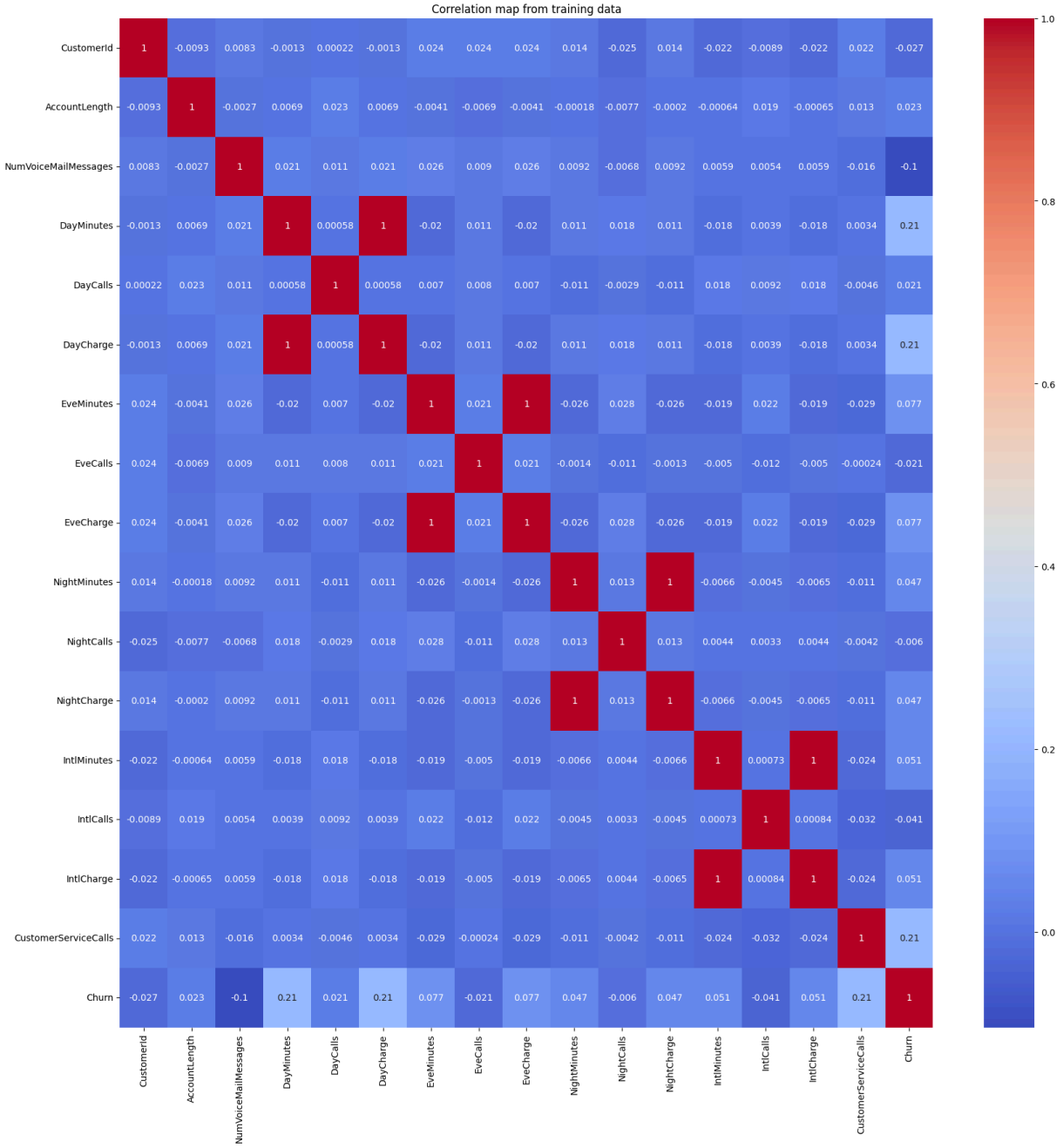
In [6]:
```
# Class distribution in the training data
print(df_train["Churn"].value_counts(normalize=True))
```

```
Churn
0    0.858571
1    0.141429
Name: proportion, dtype: float64
```

In [7]:
```
# creating a new dataframe with only numerical features for correlation map and whi
df_numeric = df_train.drop(columns=categorical_features)

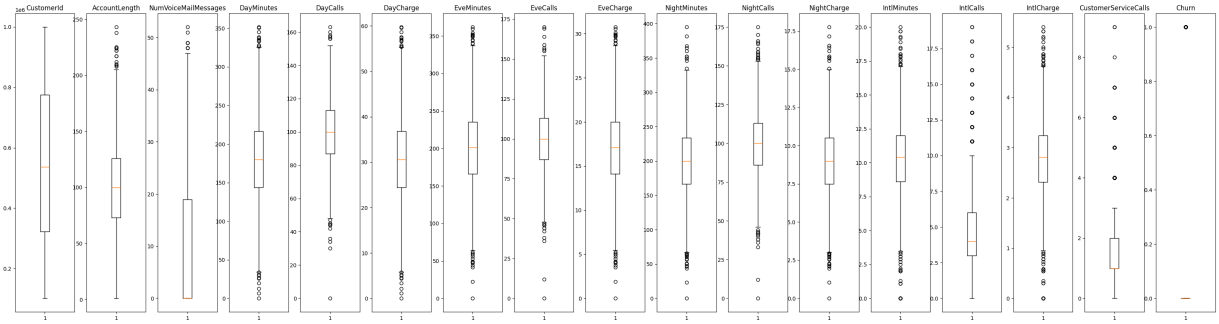
# correlation map from numerical features, modified code from lecture 12
plt.figure(figsize=(20,20))
sns.heatmap(df_numeric.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation map from training data")
```

Out[7]:
```
Text(0.5, 1.0, 'Correlation map from training data')
```



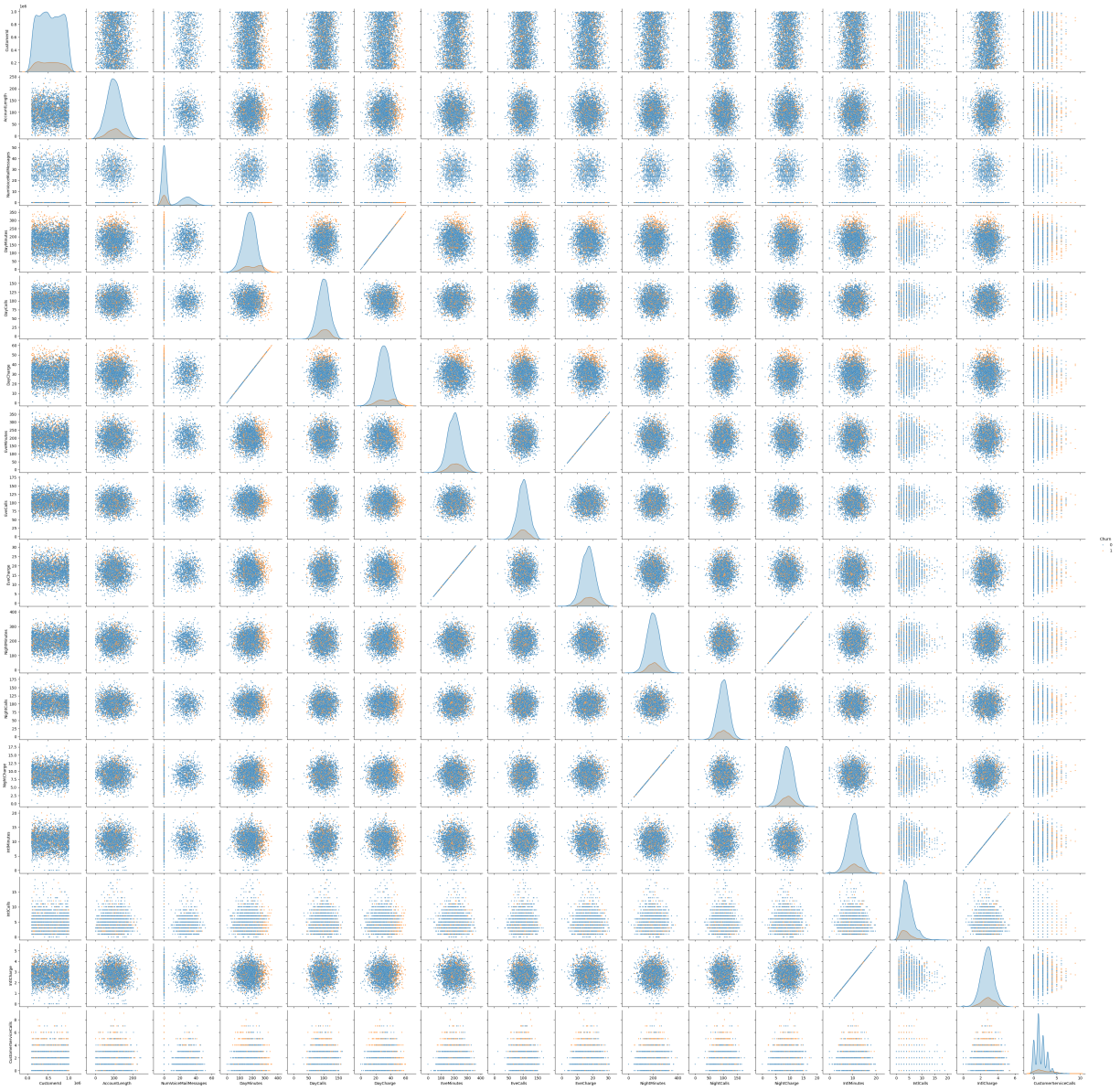
In [8]:
```
# whisker plots to check for outliers, modified code from lecture 12
fig, ax = plt.subplots(1, len(df_numeric.columns), figsize=(40,10))

for i, column in enumerate(df_numeric.columns):
    ax[i].set_title(column)
    ax[i].boxplot(df_numeric[column])
```



In [9]:
```
# Pairplots to check for relationships between features
sns.pairplot(df_numeric,  hue="Churn", plot_kws={'s':5})
```

Out[9]: <seaborn.axisgrid.PairGrid at 0x1b2a7be42b0>



Data cleaning

There is not much to do here as most cleaning will happen in the preproccesing step of the model pipeline. However we can reduce the dimensionality by removing redundant features

from the correlation map.

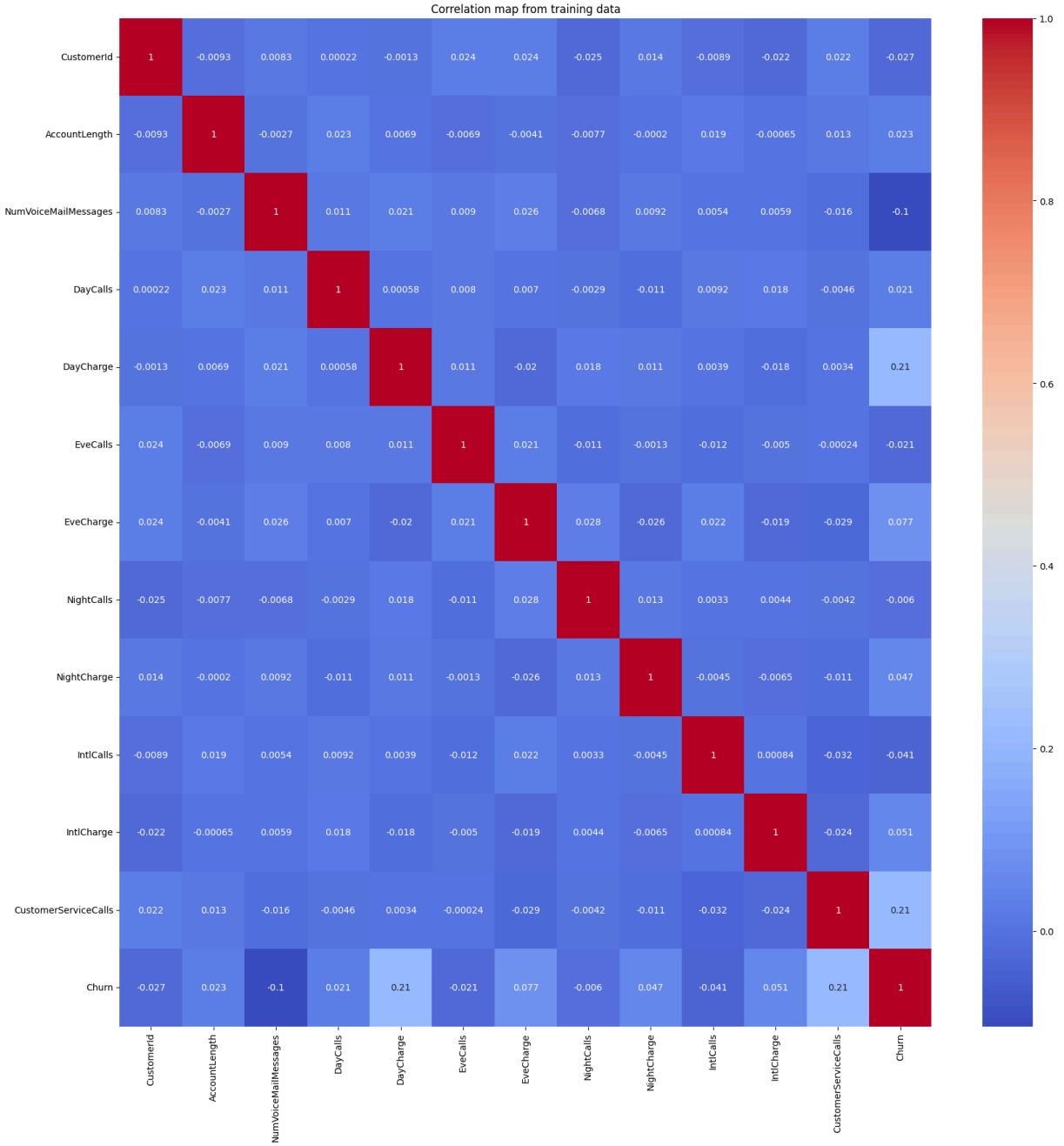
In [7]:
```
# Dropping correlated values
df_train = df_train.drop(['DayMinutes', 'EveMinutes', 'NightMinutes', 'IntlMinutes'
df_submission =df_submission.drop(['DayMinutes', 'EveMinutes', 'NightMinutes', 'Int
```

Data exploration and visualisation after cleaning

In [11]:
```
# recreating the dataframe with only numerical features (as we dropped some feature
df_numeric = df_train.drop(columns=categorical_features)

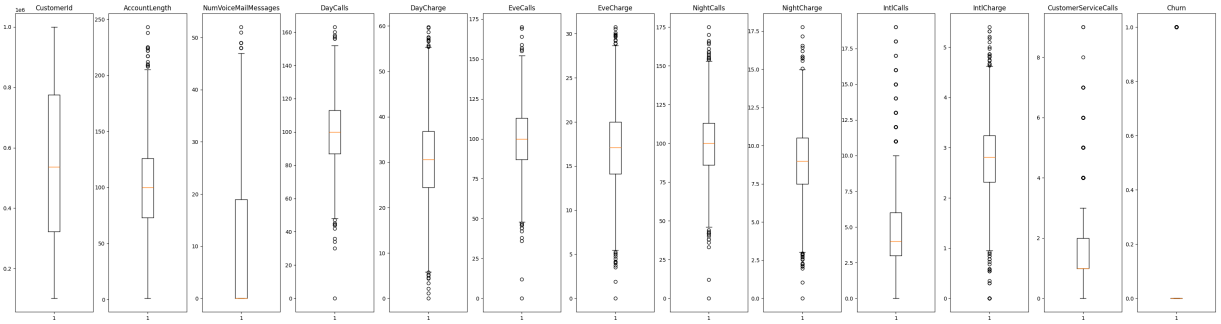
# correlation map from numerical features, modified code from lecture 12
plt.figure(figsize=(20,20))
sns.heatmap(df_numeric.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation map from training data")
```

Out[11]:
```
Text(0.5, 1.0, 'Correlation map from training data')
```



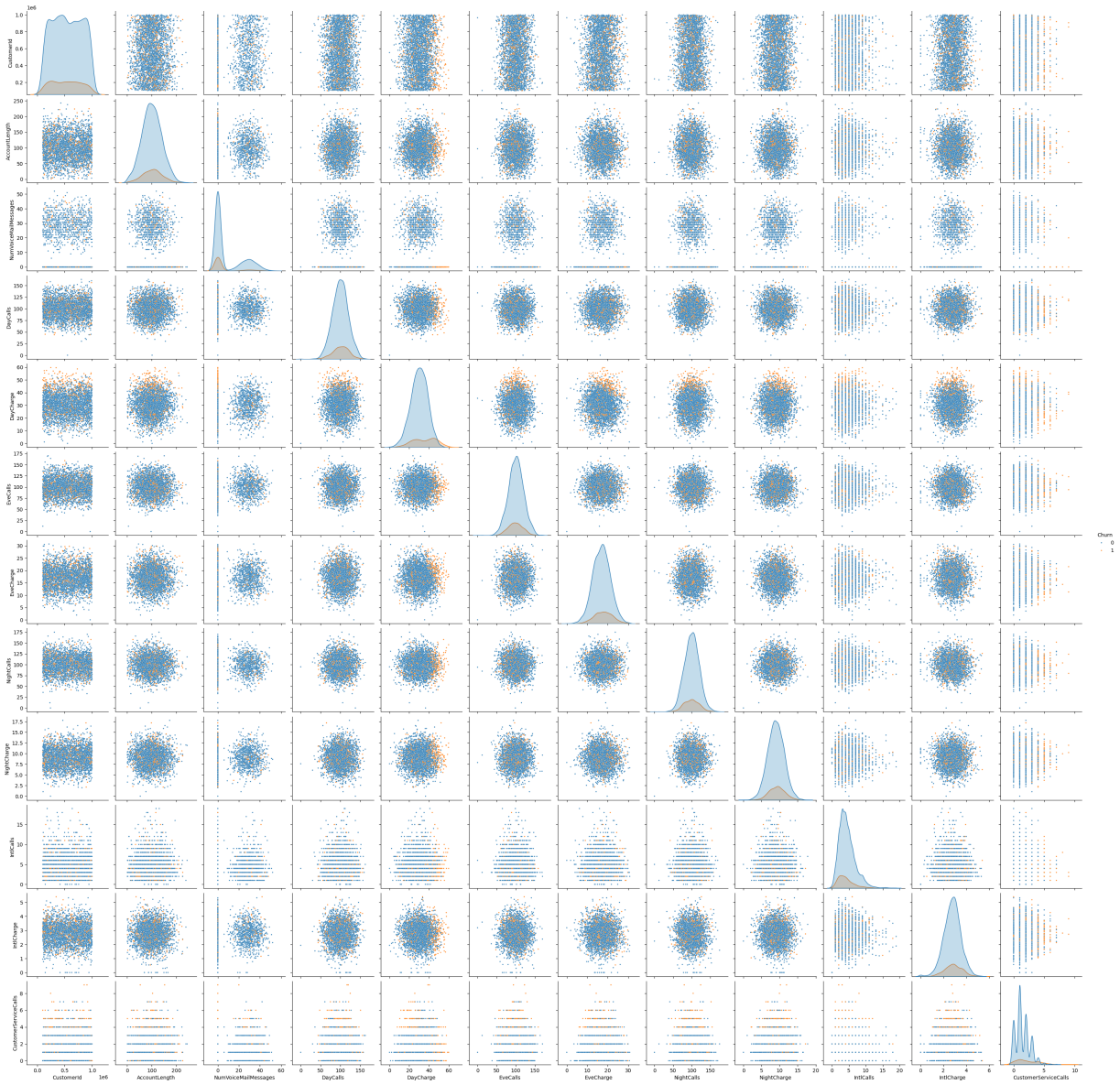
In [12]:
```
# whisker plots to check for outliers, modified code from lecture 12
fig, ax = plt.subplots(1, len(df_numeric.columns), figsize=(40,10))

for i, column in enumerate(df_numeric.columns):
    ax[i].set_title(column)
    ax[i].boxplot(df_numeric[column])
```



In [13]:
```
# Pairplots to check for relationships between features
sns.pairplot(df_numeric,  hue="Churn", plot_kws={'s':5})
```

Out[13]: <seaborn.axisgrid.PairGrid at 0x1b2bea9f700>



Modeling

Data split

In [8]:
```
x_train = df_train.drop(columns=["Churn", "CustomerId"])
y_train = df_train["Churn"]

X_train, X_test, Y_train, Y_test = train_test_split(x_train, y_train, test_size=0.4
```

General Evaluation for a model

In [9]:
```
def evaluate_model(model, name):
    # Evaluate the model on the test set and print the accuracy and F1 score
    print(f"Evaluating {name}")
    y_pred = model.predict(X_test)

    # Print scores
    print(f"accuracy: {accuracy_score(Y_test, y_pred)}")
    print(f"F1 score: {f1_score(Y_test, y_pred, average='macro')}")

    # Confusion matrix
    cm = confusion_matrix(Y_test, y_pred)

    ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=['No Churn', 'Churn']
    ).plot()

    plt.title(name)
    plt.show()
```

Preprocessing for the pipeline

In [10]:
```
# Naming the columns with categorical data for preprocessing
# (redoing this as we dropped some features)
categorical_features = [
    "State",
    "AreaCode",
    "InternationalPlan",
    "VoiceMailPlan"
]

# Getting the columns with numerical data for preprocessing
numerical_features = []
for col in X_train.columns:
    if col not in categorical_features:
        numerical_features.append(col)


# Numerical preprocessing
numerical_pipeline = Pipeline([
    # We dont have any missing data, but adding imputer for robustness as mentioned
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
```

```
])

# Numerical preprocessing with no scaling
numerical_pipeline_no_scaling = Pipeline([
    # Some models dont require scaling
    ('imputer', SimpleImputer(strategy='median')),
])

# Categorical preprocessing
categorical_pipeline = Pipeline([
    # One hot encoding because we have no ordinal features
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore', drop='first', sparse_output=
])
```

Setting up different preprocessors

In [11]:
```
# Preproccesor
preprocessor = ColumnTransformer([
    ('numerical', numerical_pipeline, numerical_features),
    ('categorical', categorical_pipeline, categorical_features)
])

# Preproccesor without scaling
preprocessor_no_scaling = ColumnTransformer([
    ('numerical', numerical_pipeline_no_scaling, numerical_features),
    ('categorical', categorical_pipeline, categorical_features)
])
```

Training models using pipelines

Preprocessing → Test and tune model hyperparameters using cross-validation and gridsearch

In [16]:
```
# Random forest classifier
rf_pipeline = Pipeline([
    ('preprocessor', preprocessor_no_scaling),
    ('classifier', RandomForestClassifier(random_state=42))
])

rf_param_grid = {
    'classifier__n_estimators': [50, 100, 200],
    'classifier__max_depth': [3, 5, 7, None],
    'classifier__min_samples_split': [2, 5, 10],
    'classifier__min_samples_leaf': [1, 2, 4]
}

# Apply pipeline to grid search with cross-validation
rf_grid = GridSearchCV(
    rf_pipeline,
    param_grid=rf_param_grid,
    cv=5,
```

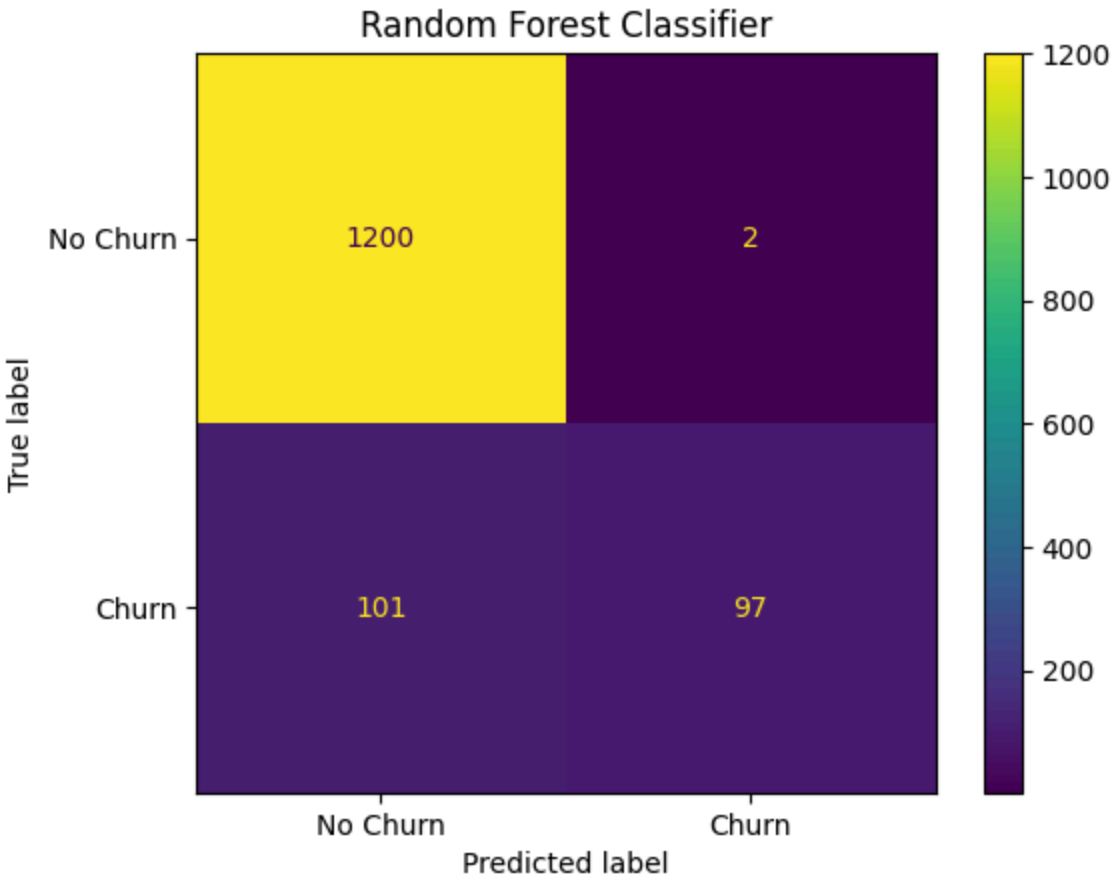
```
    scoring='f1_macro'
)

# Fit the grid search to the training data
rf_grid.fit(X_train, Y_train)
print(rf_grid.best_score_)
RF_best_model = rf_grid.best_estimator_
```

```
0.8052169018324319
```

In [17]:
```
evaluate_model(RF_best_model, "Random Forest Classifier")
```

```
Evaluating Random Forest Classifier
accuracy: 0.9264285714285714
F1 score: 0.8060240169708808
```



In [18]:
```
# LogisticRegression
lr_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('pca', PCA()),
    ('classifier', LogisticRegression(max_iter=1000, random_state=42))
])

lr_param_grid = [
    {
        'pca__n_components': [0.95, 5, 10, 15],
        'classifier__C': [0.001, 0.01, 0.1, 1, 10, 100],
        'classifier__penalty': ['l1', 'l2'],
        'classifier__solver': ['liblinear', 'saga']
    },
```

```
    {
        'pca__n_components': [0.95, 5, 10, 15],
        'classifier__C': [0.001, 0.01, 0.1, 1, 10, 100],
        'classifier__penalty': ['elasticnet'],
        'classifier__solver': ['saga'],
        'classifier__l1_ratio': [0.25, 0.5, 0.75]
    }
]

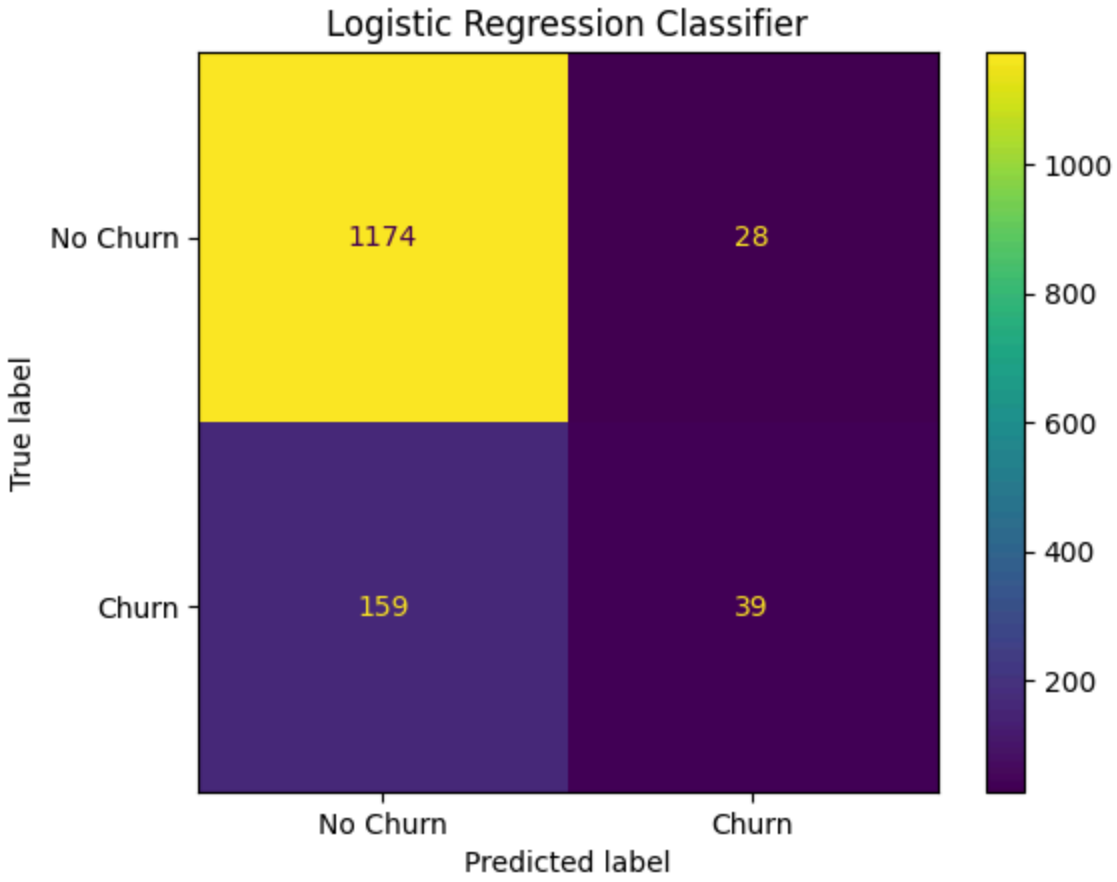
lr_grid = GridSearchCV(
    lr_pipeline,
    param_grid=lr_param_grid,
    cv=5,
    scoring='f1_macro'
)

# Fit the grid search to the training data
lr_grid.fit(X_train, Y_train)
print(lr_grid.best_score_)
LR_best_model = lr_grid.best_estimator_
```

```
0.6346979384328713
```

In [19]:
```
evaluate_model(LR_best_model, "Logistic Regression Classifier")
```

```
Evaluating Logistic Regression Classifier
accuracy: 0.8664285714285714
F1 score: 0.6102861821294332
```



In [20]:
```
# SVC model
svc_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('pca', PCA()),
    ('classifier', SVC(random_state=42))
])

svc_param_grid = [
    {
        "pca__n_components": [2, 5, 10, 15],
        'classifier__C': [0.1, 1, 10],
        'classifier__kernel': ['linear']
    },
    {
        "pca__n_components": [10, 12, 14, 15, 16],
        'classifier__C': [0.1, 1, 10],
        'classifier__kernel': ['rbf'],
        'classifier__gamma': ['scale', 'auto']
    }
]

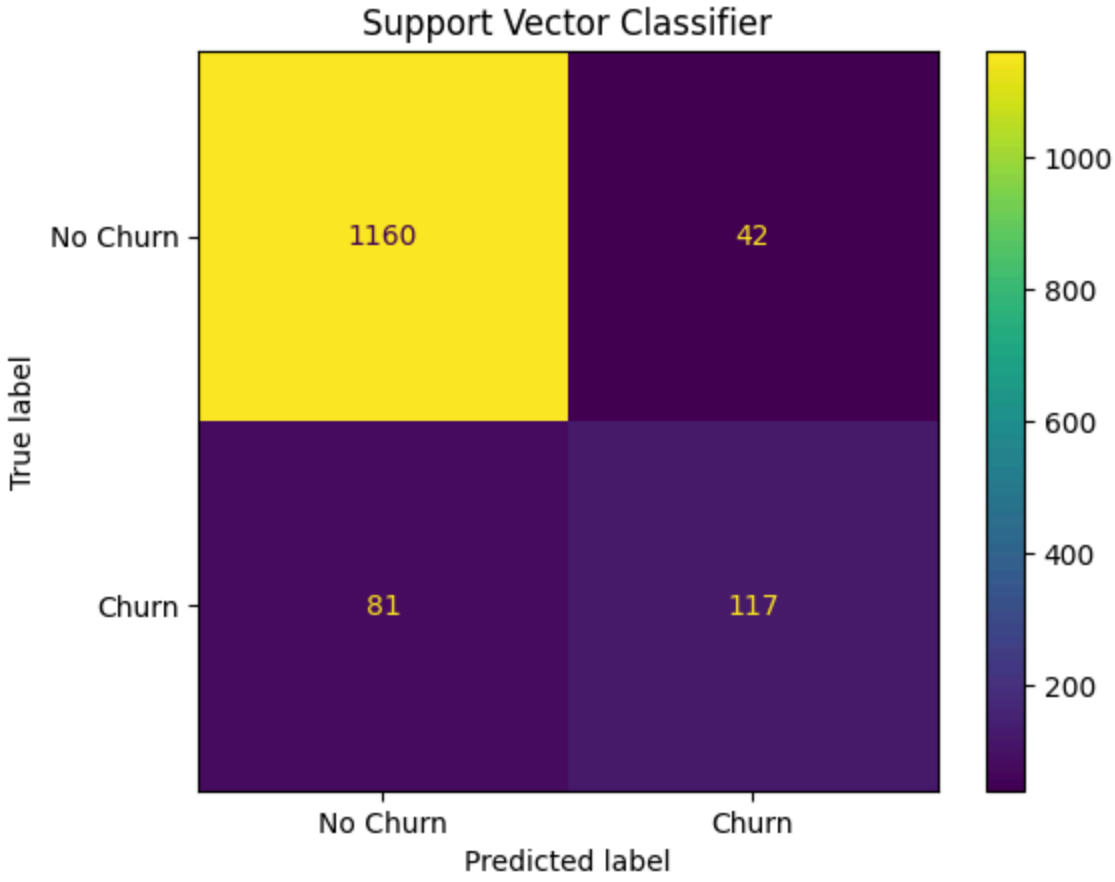
svc_grid = GridSearchCV(
    svc_pipeline,
    param_grid=svc_param_grid,
    cv=5,
    scoring='f1_macro'
)

# Fit the grid search to the training data
svc_grid.fit(X_train, Y_train)
print(svc_grid.best_score_)
SVC_best_model = svc_grid.best_estimator_
```

```
0.81413212238249
```

In [21]:
```
evaluate_model(SVC_best_model, "Support Vector Classifier")
```

```
Evaluating Support Vector Classifier
accuracy: 0.9121428571428571
F1 score: 0.8025571260022634
```



In [23]:
```python
# Gradient Boost

gb_pipeline = Pipeline([
    ('preprocessor', preprocessor_no_scaling),
    ('classifier', GradientBoostingClassifier(random_state=42))
])

gb_param_grid = {
    'classifier__n_estimators': [50, 100, 200],
    'classifier__learning_rate': [0.001, 0.01, 0.1],
    'classifier__max_depth': [3, 5, 7],
    'classifier__subsample': [0.7],
    'classifier__min_samples_split': [10]
}

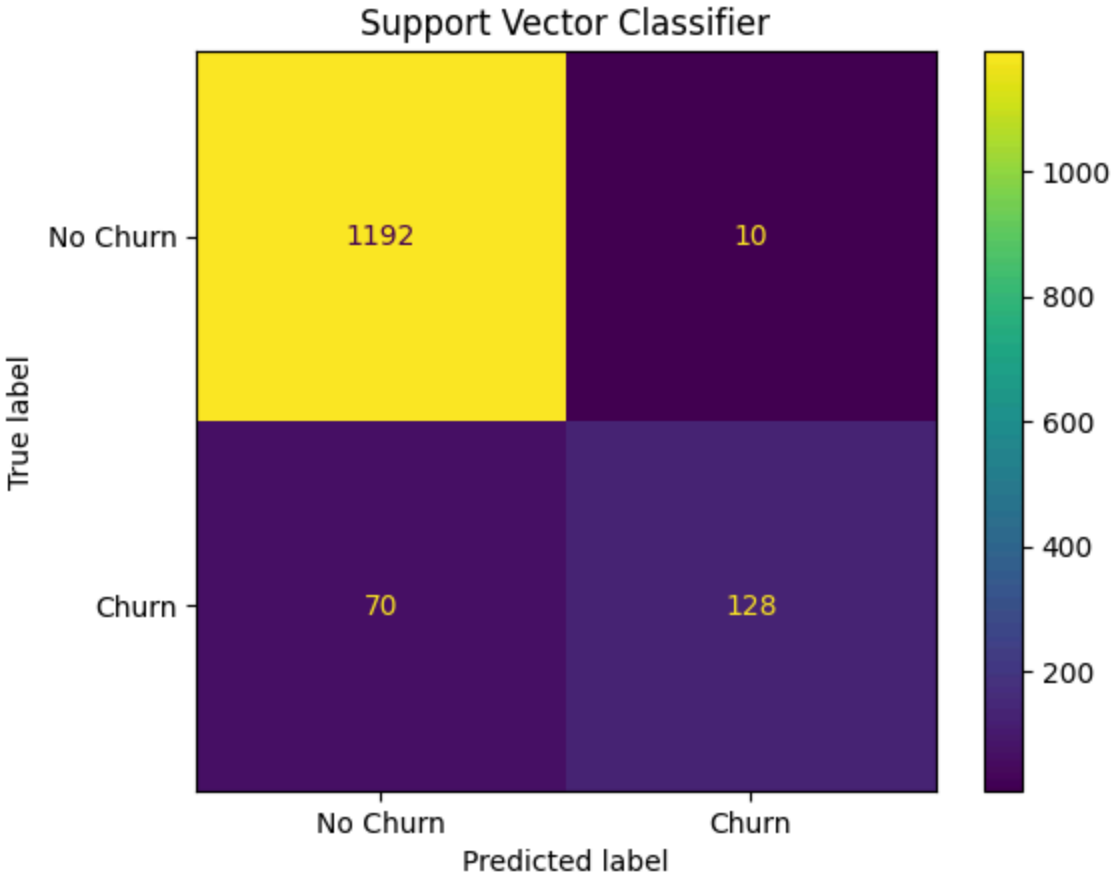
gb_grid = GridSearchCV(
    gb_pipeline,
    param_grid=gb_param_grid,
    cv=5,
    scoring='f1_macro'
)

# Fit the grid search to the training data
gb_grid.fit(X_train, Y_train)
print(gb_grid.best_score_)
gb_best_model = gb_grid.best_estimator_
```

```
0.8979199791219893
```

In [24]:
```
evaluate_model(gb_best_model, "Support Vector Classifier")
```

```
Evaluating Support Vector Classifier
accuracy: 0.9428571428571428
F1 score: 0.8647186147186148
```



In [12]:
```
# XGBoost

xgb_pipeline = Pipeline([
    ('preprocessor', preprocessor_no_scaling),
    ('classifier', XGBClassifier(random_state=42))
])

xgb_param_grid = {
    'classifier__n_estimators': [20, 50, 100, 200],
    'classifier__learning_rate': [0.001, 0.01, 0.1],
    'classifier__max_depth': [3, 5, 7],
    'classifier__subsample': [0.7],
    'classifier__gamma': [0, 3, 5, 10],
    'classifier__reg_lambda': [0, 3, 5, 10],
}

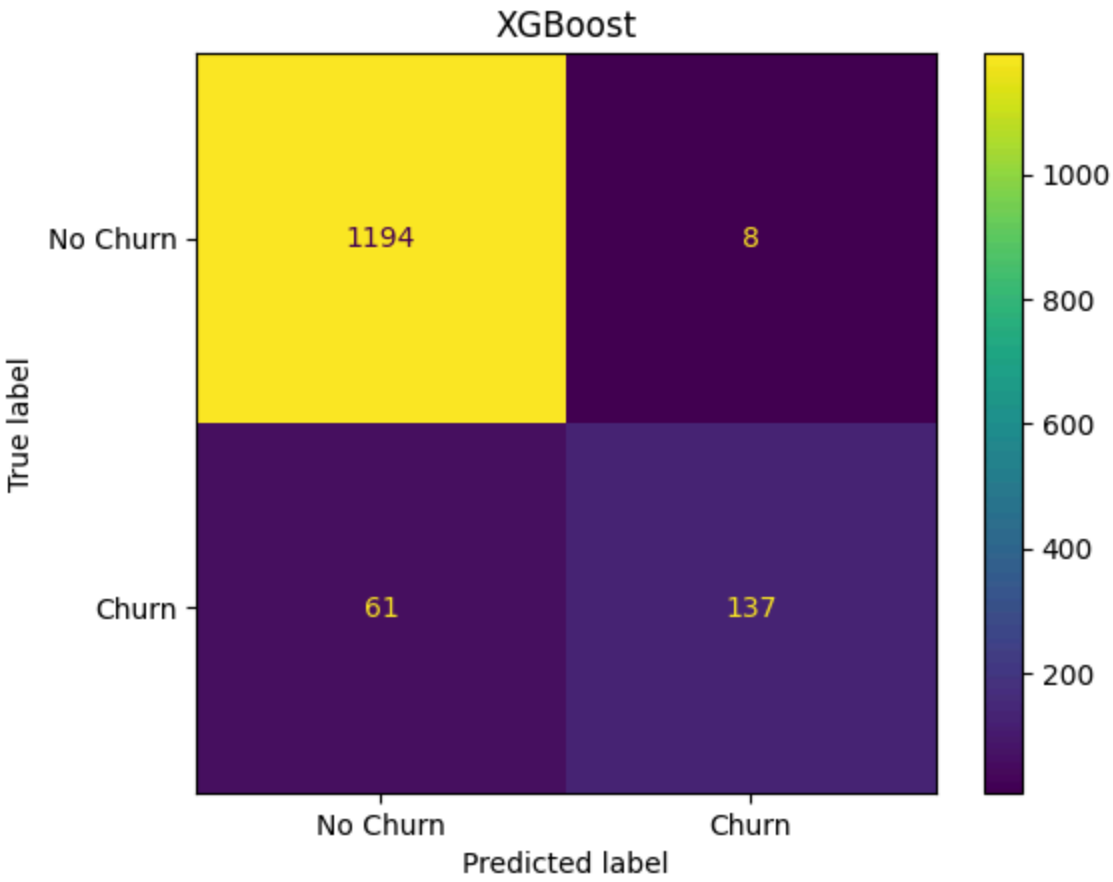
xgb_grid = GridSearchCV(
    xgb_pipeline,
    param_grid=xgb_param_grid,
    cv=5,
    scoring='f1_macro'
)
```

```
# Fit the grid search to the training data
xgb_grid.fit(X_train, Y_train)
print(xgb_grid.best_score_)
xgb_best_model = xgb_grid.best_estimator_
```

```
0.8965496027126392
```

In [14]:
```
evaluate_model(xgb_best_model, "XGBoost")
```

```
Evaluating XGBoost
accuracy: 0.9507142857142857
F1 score: 0.8853753955794772
```



Kaggle submission

Until now we have tested every model on the test set to see performance and confusion matrix. We can not use these metrics to select our final model as it would mean that the test set has influenced the model choice. Therefore we compare the best scores from the grid search

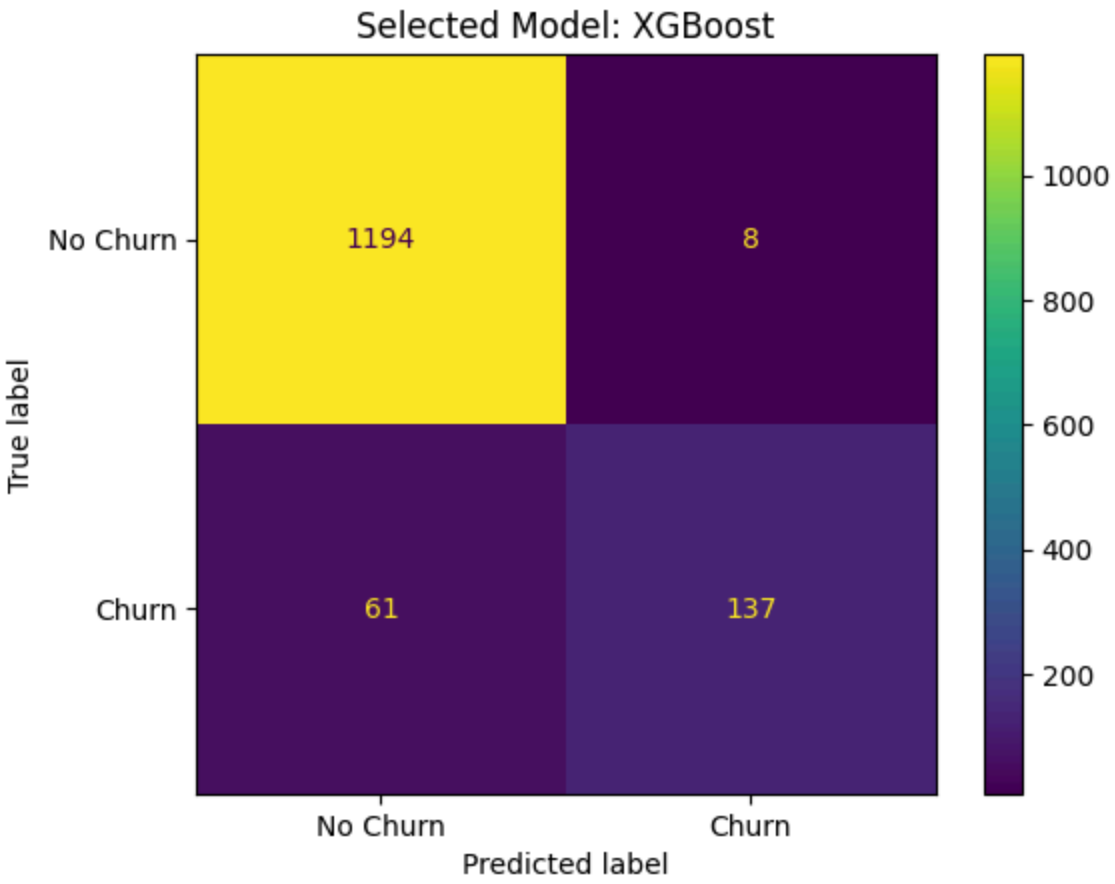
In [25]:
```
print(f"SVC model score: {svc_grid.best_score_}")
print(f"Random Forest model score: {rf_grid.best_score_}")
print(f"Logistic Regression model score: {lr_grid.best_score_}")
print(f"Gradient Boost model score: {gb_grid.best_score_}")
print(f"XGBoost model score: {xgb_grid.best_score_}")
```

```
SVC model score: 0.81413212238249
Random Forest model score: 0.8052169018324319
Logistic Regression model score: 0.6346979384328713
Gradient Boost model score: 0.8979199791219893
XGBoost model score: 0.8965496027126392
```

When testing Gradient Boost as the final model, it became clear that the model was overfitted, so XGBoost was chosen instead.

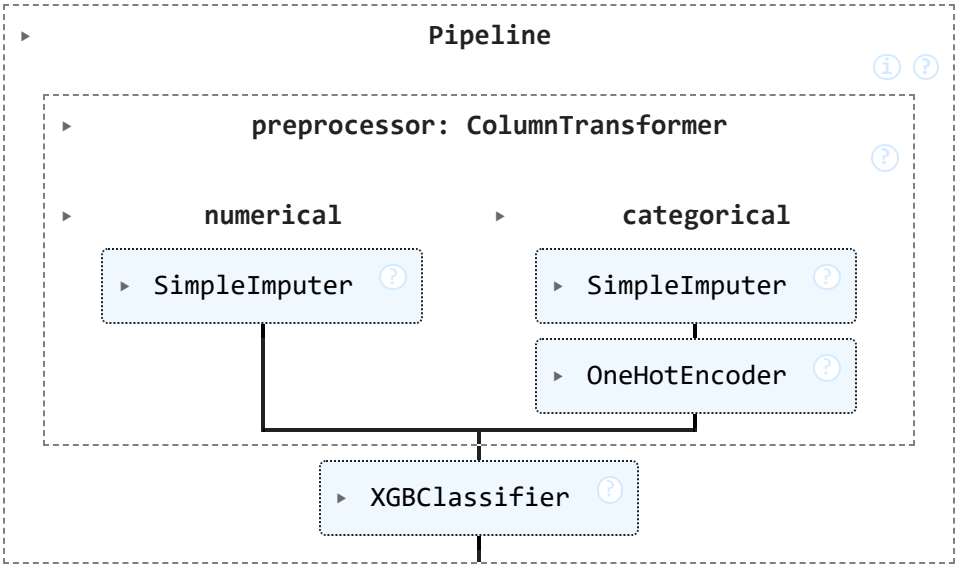
In [26]:
```
# Select model and evaluate it on the test set
model = xgb_best_model
evaluate_model(model, "Selected Model: XGBoost")
```

```
Evaluating Selected Model: XGBoost
accuracy: 0.9507142857142857
F1 score: 0.8853753955794772
```



In [27]:
```
# Train model on all available data
model.fit(x_train, y_train)
```

Out[27]:



In [28]:
```
# Get submission data and customer ID
customerIds = df_submission['CustomerId']
x_input = df_submission.drop(['CustomerId'], axis=1)

# Predict targets
y_pred = model.predict(x_input)

# create submission dataframe and save as CSV file
submission = pd.DataFrame({
    "CustomerId": customerIds,
    "Churn": y_pred
})
submission.to_csv("submissionCA4.csv", index=False)
```

In [29]:
```
# Check class distribution in the submission file
print(submission["Churn"].value_counts(normalize=True))
```

```
Churn
0    0.878667
1    0.121333
Name: proportion, dtype: float64
```